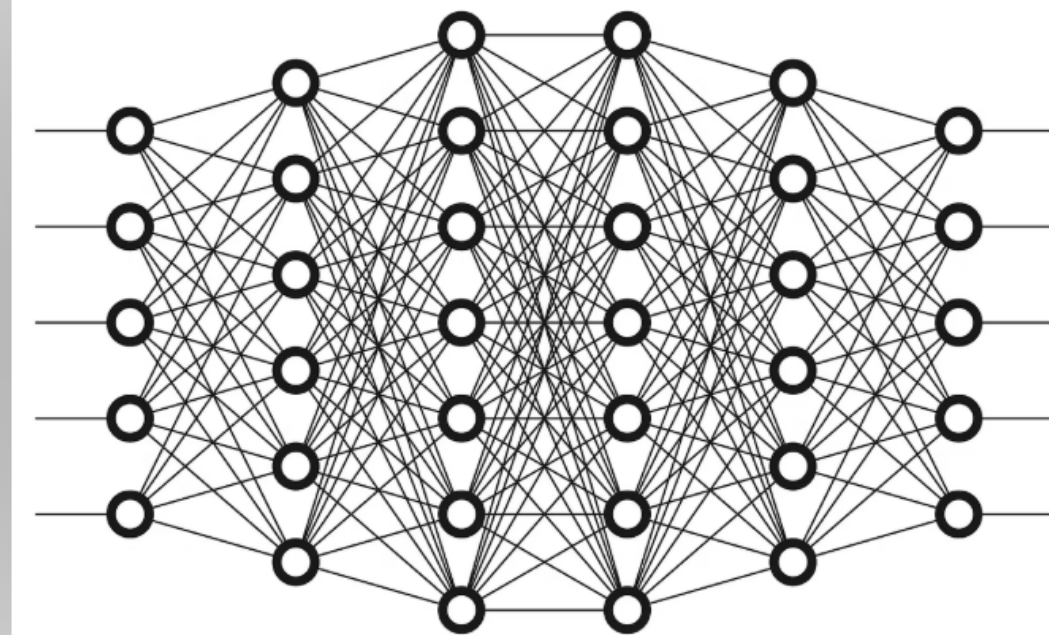




AI Academy

From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems



Math4AI Capstone Project

From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems

Authors: Ferman Khankishiyev, Mahmud Ramazanov, Hesen Mammadov, Bayram Bayramov

Course: National AI Academy

Date: 04.04.2026

Team Member	Role	Area of Responsibility
Ferman Khankishiyev	Team Lead, Lead Developer & Codebase Supervisor	Designed the initial architecture and code skeleton of the project, managed the GitHub repository, coordinated the development process and monitored the overall progress of the project and additionally improved the code
Bayram Bayramov	Data, Code & Model Developer & Documentation Lead	Conducted data research, enhanced and expanded the base code structure, contributed to the development of additional models and features, and played a significant role in the preparation of project documentation, presentations and reports
Hasan Mammadov	Repository & Integration Engineer & Code Developer	Worked on understanding extended datasets, supported repository integration, and handled debugging and overall repository maintenance
Mahmud Ramazanov	Documentation & Code Developer & Theoretical Overview	Focused on theoretical aspects, contributed to the preparation of presentations and project reports, played role in overall review and quality assurance, and made significant cotributions to code development and improvement

The Central Question



We focus on:

- *When does nonlinearity actually help?*
- *Where does additional model complexity fail to justify itself?*

The Central Question

When does a one-hidden-layer nonlinear classifier improve upon a linear decision rule, and when is added complexity unnecessary?

Why this matters:

- Complex models waste resources where simpler ones suffice
- Linear models leave performance gains unrealized when nonlinearity is needed
- Understanding this trade-off guides practical model selection

Our approach:

- Implement both models from scratch in NumPy
- Controlled comparison: same optimizer, regularization, data splits
- Test on datasets with known geometric structure

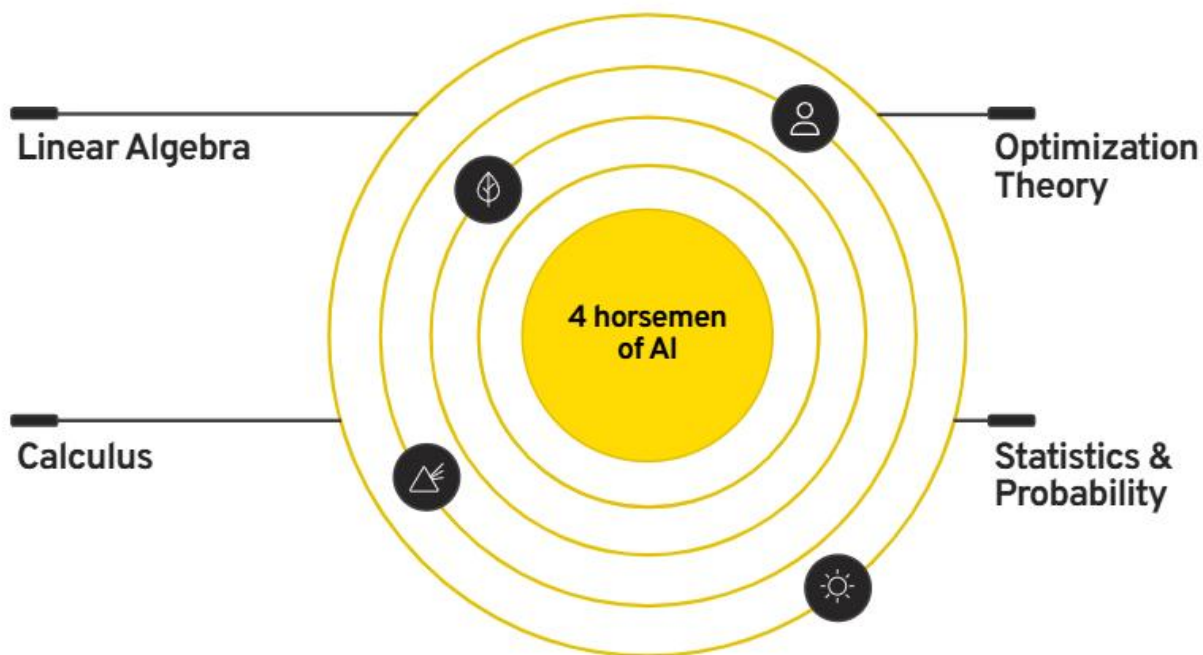
Hypotheses

Three hypotheses tested:

Hypothesis	Conclusion	Test Dataset
Linear Sufficiency	Both models perform comparably on linearly separable data	Gaussian blobs
Nonlinear Necessity	NN significantly outperforms softmax on nonlinearly separable data	Moons
Capacity Scaling	Performance improves with hidden width but with diminishing returns	Moons ablation

Mathematical Background

Needed Theory



These 4 concepts are used everywhere. But specifically,

- Linear Algebra - Track A (PCA).
- Calculus & Optimization - optimizers (SGD, Momentum, Adam)
- Statistics & Probability - repeated seed evaluation & Softmax

Dataset Overview

Dataset	Samples	Features	Classes	Train/Val/Test
Digits	1,797	64 (8×8)	10 (0-9)	1,074 / 355 / 368
Gaussian	400	2	2 (0,1)	240 / 80 / 80
Moons	400	2	2 (0,1)	240 / 80 / 80

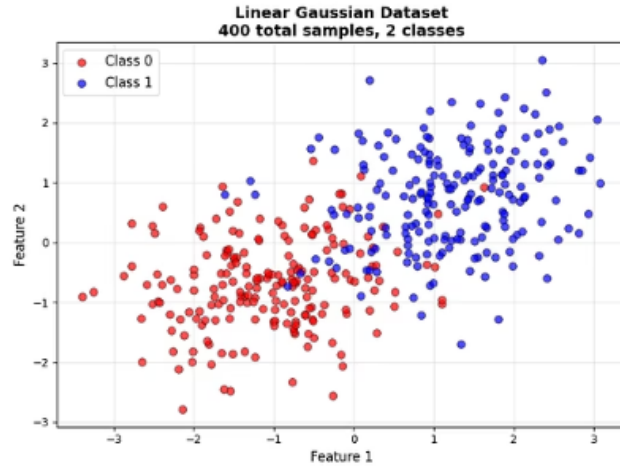
Digits notes: Grayscale, pixel range [0,1], mean intensity 0.305, well-balanced classes

Key Takeaways

- **Digits:** 1,797 handwritten digits (0-9), 8×8 pixels, normalized intensity [0,1]
- **Gaussian & Moons:** 400 samples each, 2D features, binary classification
- **Consistent split:** 60% train / 20% validation / 20% test across all datasets
- **Well-balanced:** No significant class imbalance issues

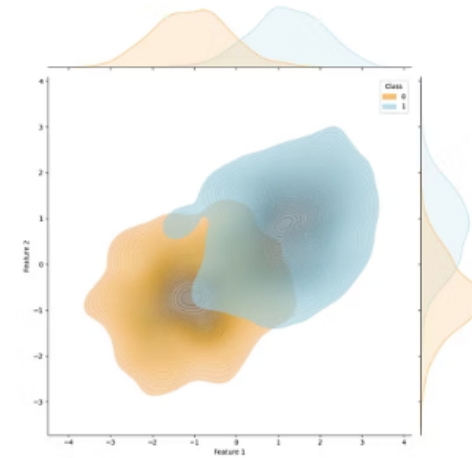
As for the experimental procedure, the evaluation protocol for both models are the same, they use same train/val/test, and the parameters remain unchanged for comparison.

Linear Gaussian Data Visualization



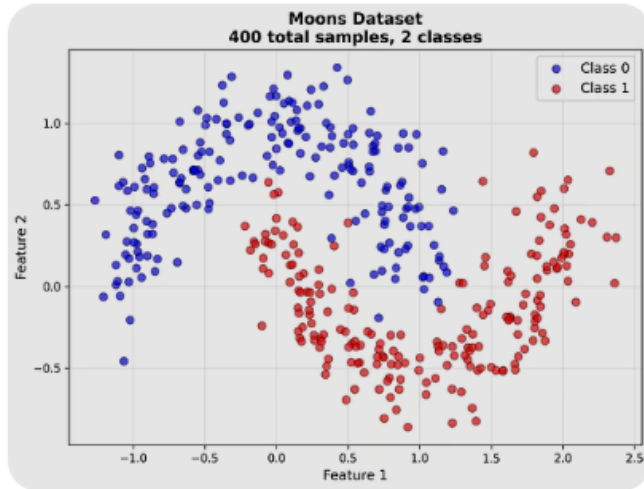
Scatter

Looking at the Linear Gaussian visualization, we see two clearly separated clusters. Class 0, shown in red, forms a tight Gaussian distribution centered on negative x-values. Class 1, in blue, is similarly compact but centered on positive x-values. There's very little overlap between the classes, and the decision boundary is essentially a 135 degree line through the origin.

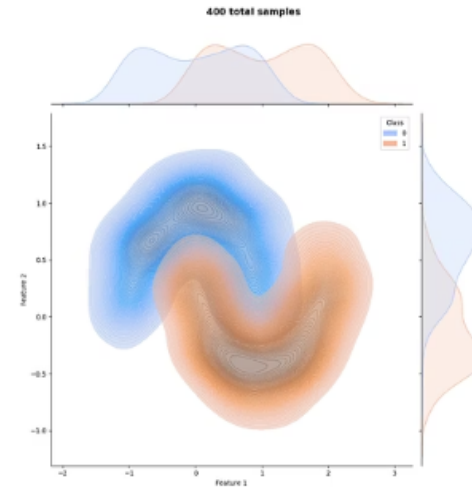


Density

Moons Data Visualization



Scatter



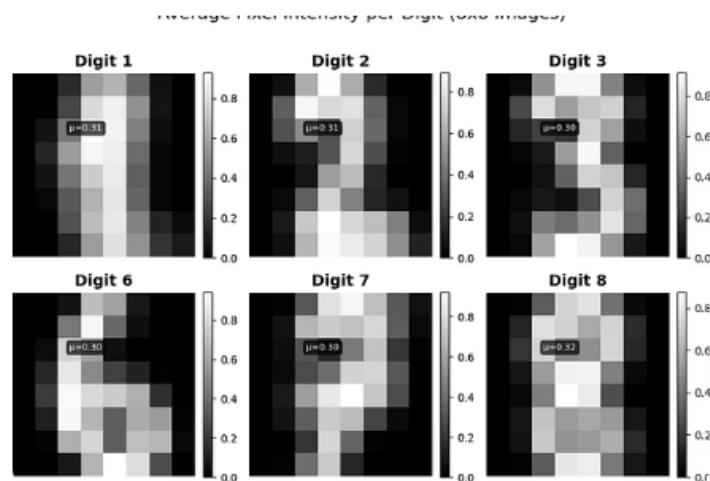
Density

The Moons dataset presents two non-linearly separable, interlocking crescent-shaped clusters that cannot be separated by any straight line, forcing models to learn curved decision boundaries to achieve good classification.

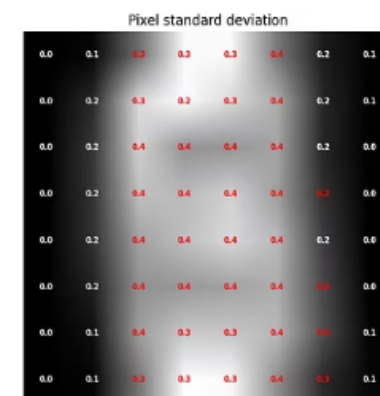
Digits data visualizations



10 random examples per digit 0-9. Digits 0,1,7 are consistent; digits 3,5,8,9 show more variation in slants and loops.



Prototype images per digit. Light pixels show consistent activation.



Edges and loops vary most (handwriting differences); centers are stable.

Models Compared

Why the comparison is fair?

Softmax	One-Hidden-Layer Neural Network
$P(y \mid x) = softmax(Wx + b)$	$h = tanh(W_1x + b_1)$
Decision boundaries = hyperplanes	$P = softmax(W_2h + b_2)$
Partitions space into convex polytopes	Hidden layer warps input space
Cannot represent curved boundaries	Can learn curved boundaries via nonlinear activation

Key Insight: This comparison is fair because both models are trained with the same optimizer, regularization, data splits, training procedure, softmax output layer, loss function, and hyperparameters. The only architectural difference is the hidden layer, which adds a nonlinear transformation of the input. That means any performance gap can be attributed to the hidden layer's capacity to learn nonlinear structure, rather than to differences in training setup or output behavior.

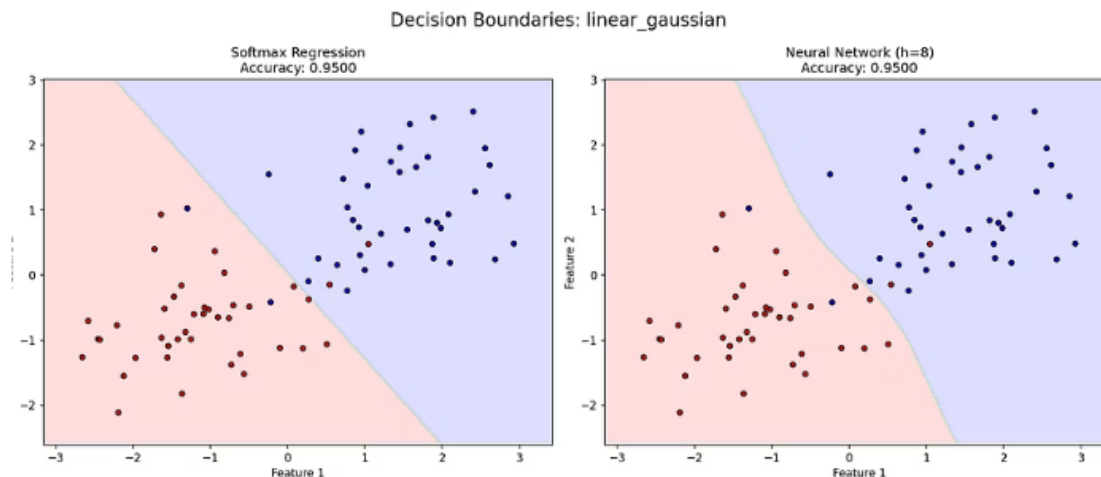
Implementation Sanity Checks

Validation of Correct Implementation:

Check	Result	Why we need
Gradient verification (finite differences)	Relative error $< 10^{-4}$	Correct numerical computation
Probability sum check	$\sum p_j = 1$ for all batches	Axioms of probability
NaN/Inf detection	None detected	Integer overflow/underflow
Loss decrease on tiny subset (50 samples)	Near-zero loss by epoch 200	Implementations works as expected
Overfitting on small subset	Training accuracy $>$ validation accuracy	Model has learning capacity

Linear Gaussian Results

Both models achieve identical performance



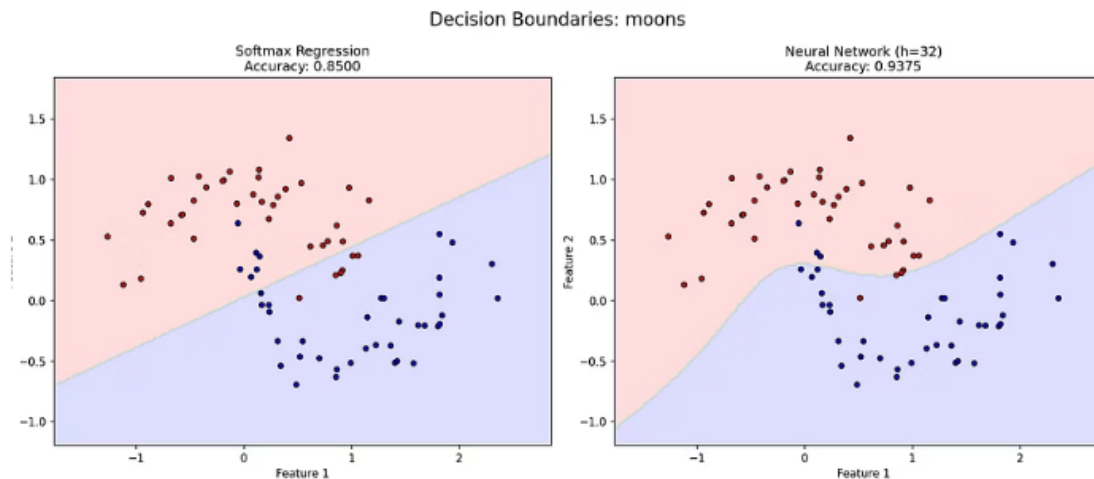
Model	Test Accuracy
Softmax Regression	95.0%
Neural Network	95.0%

Conclusion:

The hidden layer adds unnecessary complexity with no measurable benefit.

Therefore, linear classifier is sufficient.

Moons Results – The Power of Nonlinearity



Model	Test Accuracy
Softmax Regression	85.0%
Neural Network ($h = 8$)	95.0%

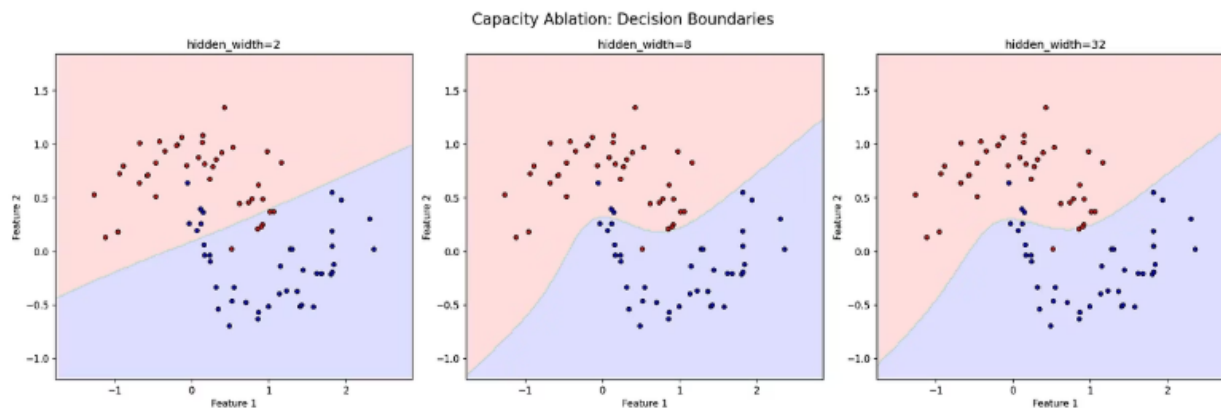
+10 percentage point improvement !

Conclusion:

- Curved boundaries **require** nonlinear classifiers
- Hidden layer transforms the input space to enable linear separability
- The representational change is essential

Capacity Ablation on Moons

Hidden width h progression:



h	Validation Loss	Test Accuracy	Status
2	0.230	85.0%	Under-capacity → linear performance
8	0.159	95.0%	Optimal – captures true geometry
32	0.163	93.75%	Excess capacity → diminishing returns

Key Insight: There's a **hard capacity threshold** – below it, the network cannot learn curved boundaries regardless of training

Failure Case

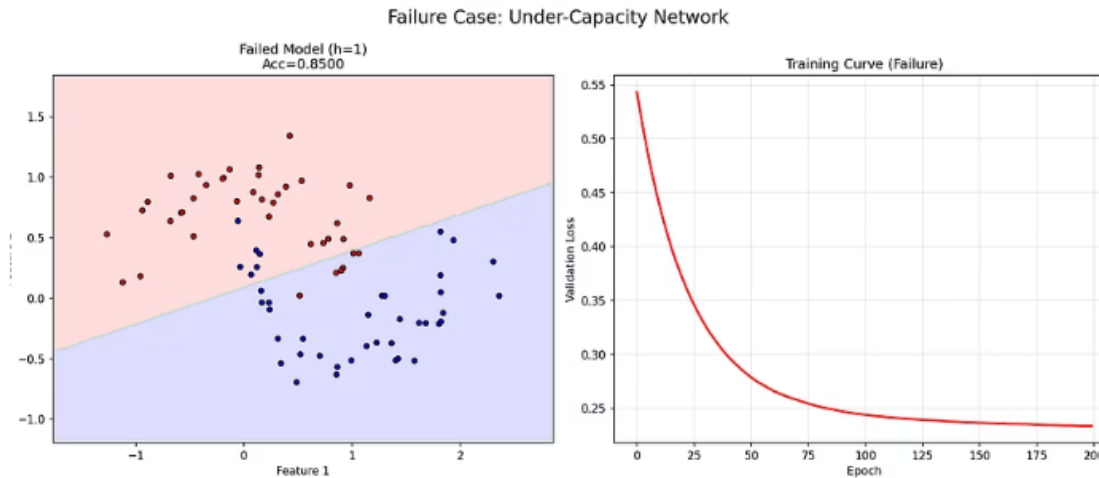
Hidden width = 1 $\rightarrow$ collapses to linear performance

Mathematical explanation:

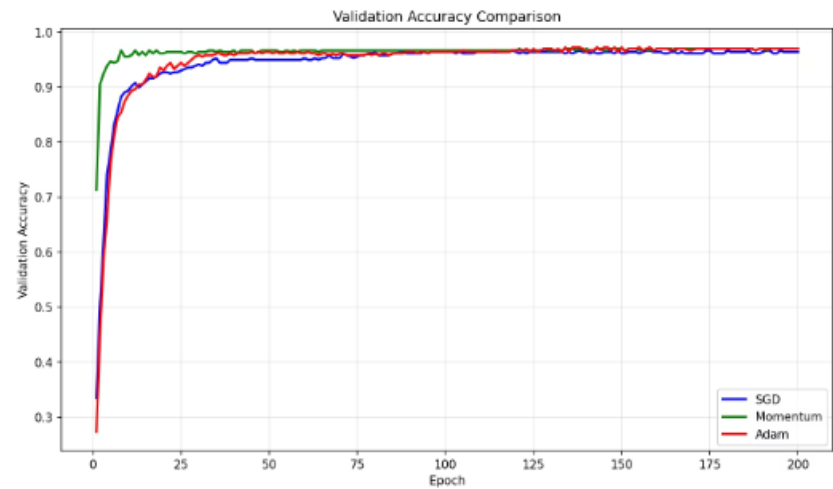
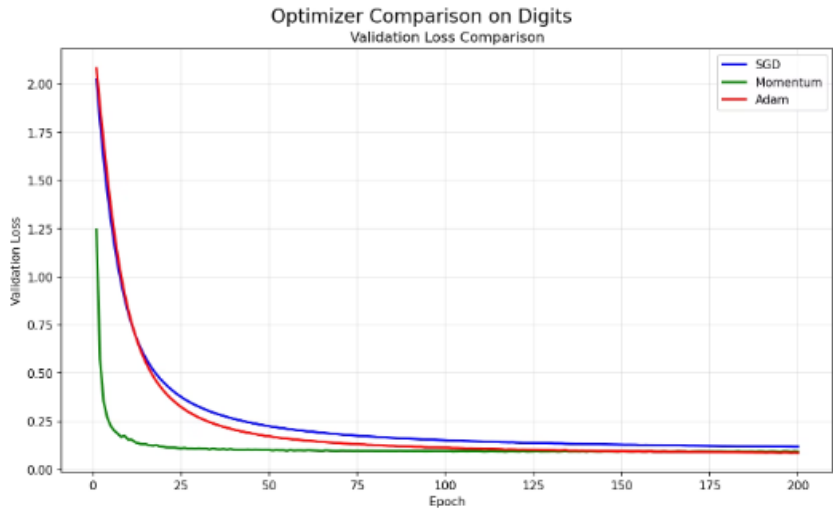
$$s = w_2 \times \tanh(w_1^\top x + b_1) + b_2$$

- Composition of scalar tanh with affine map
- Depends only on **single projection** of input
- Cannot represent 2D curved boundaries
- Condition number diverges (W_1 rank-1)

Insight: Capacity thresholds depend on **geometric properties**, not just parameter counts



Optimizer Study on Digits



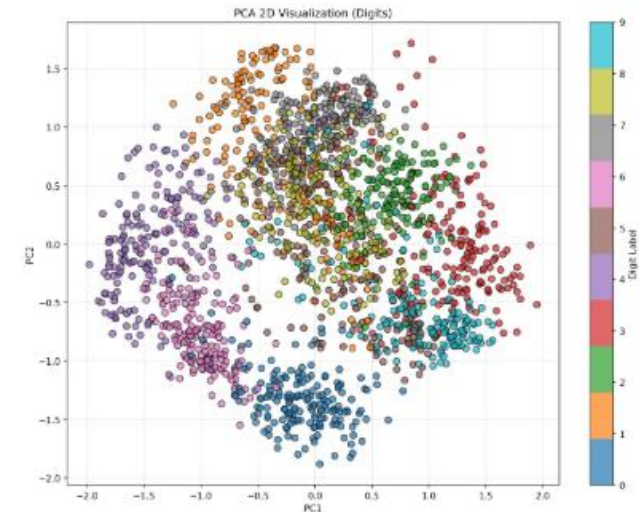
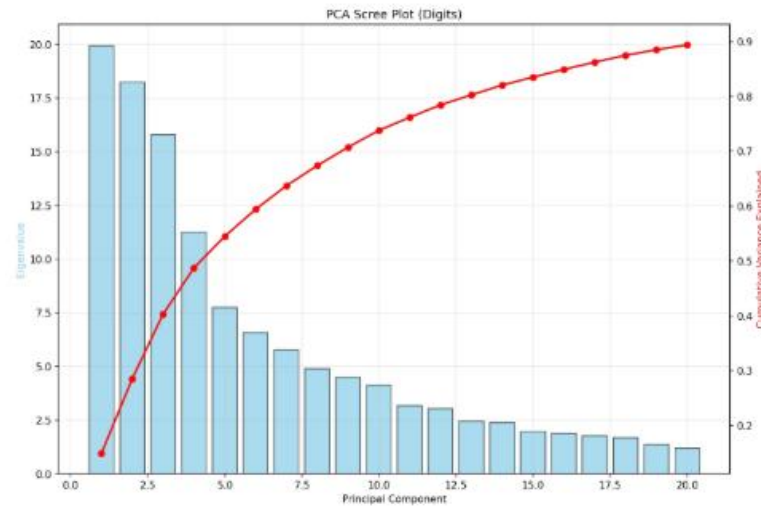
Optimizer	Validation Accuracy
SGD	0.9634
Momentum	0.9690
Adam	0.9690

Conclusion:

- Adam converges stable, Momentum converges early and quickly, SGD converges in medium pace, but is more unstable
- All optimizers produce good results
- Loss landscape has broad basin of good solutions

Why this way? Because, SGD uses only local knowledge. In that way, SGD explores loss landscape erratically. It may be quick, but it is highly likely to stuck in local minima. Momentum uses previous knowledge of gradients, hence accumulating velocity, which gives more information about previous gradients, giving more chance to explore loss landscape. Adam on the other hand, Adaptive Moment Optimization uses RMSProp and Momentum, using both previous gradients and gradients per parameter, giving chance to optimize more "optimally". In this case, SGD is baseline. Momentum outperforms Adam, because of the little data. On noisy and complex problems, Adam wins most of the time.

PCA Analysis of Digits



Explained variance:

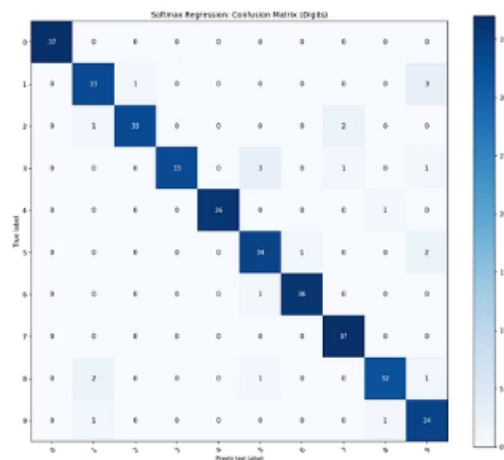
- First 10 components: ~65% of variance
- First 40 components: ~85% of variance
- Elbow around 10-20 components → low effective dimensionality

Dimensions (k)	Test Accuracy	Test Cross-Entropy
10	89.95%	0.3586
20	92.66%	0.2880
40	93.21%	0.2714
64 (full)	93.75%	0.2698

Reducing 64 → 40 preserves 99.4% of original accuracy

Confusion Matrix – Softmax Regression

Softmax Regression: Confusion Matrix on Digits Test Set



Metric

Value

Overall Accuracy

93.8%

Key Observations:

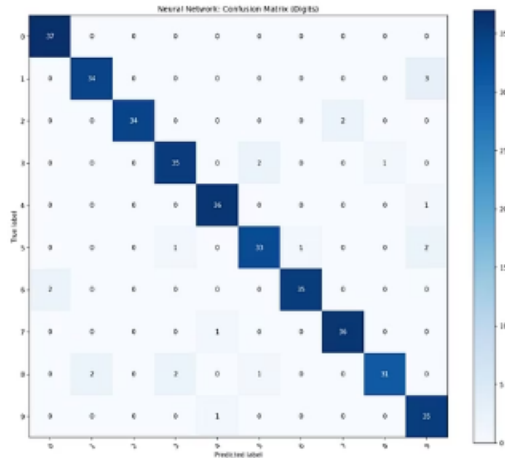
- Most errors occur between **visually similar digits**
- Common misclassifications: 1 ↔ 9, 2 ↔ 7, 3 ↔ 5
- Digit 0 is perfectly classified

Interpretation:

Linear model struggles when digit shapes have overlapping features. The decision boundaries are hyperplanes in pixel space, which cannot perfectly separate digits with similar visual structure.

Confusion Matrix – Neural Network

One-Hidden-Layer Neural Network (h=32): Confusion Matrix on Digits Test Set



Metric

Value

Overall Accuracy

95.3%

Key Observations:

- Improved classification across most digits
- Notable reductions in misclassifications: 3 → 5
- Digit 0 remains perfectly classified

Interpretation:

The hidden layer learns a **nonlinear transformation** of the input space. Similar digits become more separable in the hidden representation, reducing confusion between visually ambiguous pairs.

Digits Benchmark Results

8×8 handwritten digits (10 classes, 64 dimensions)

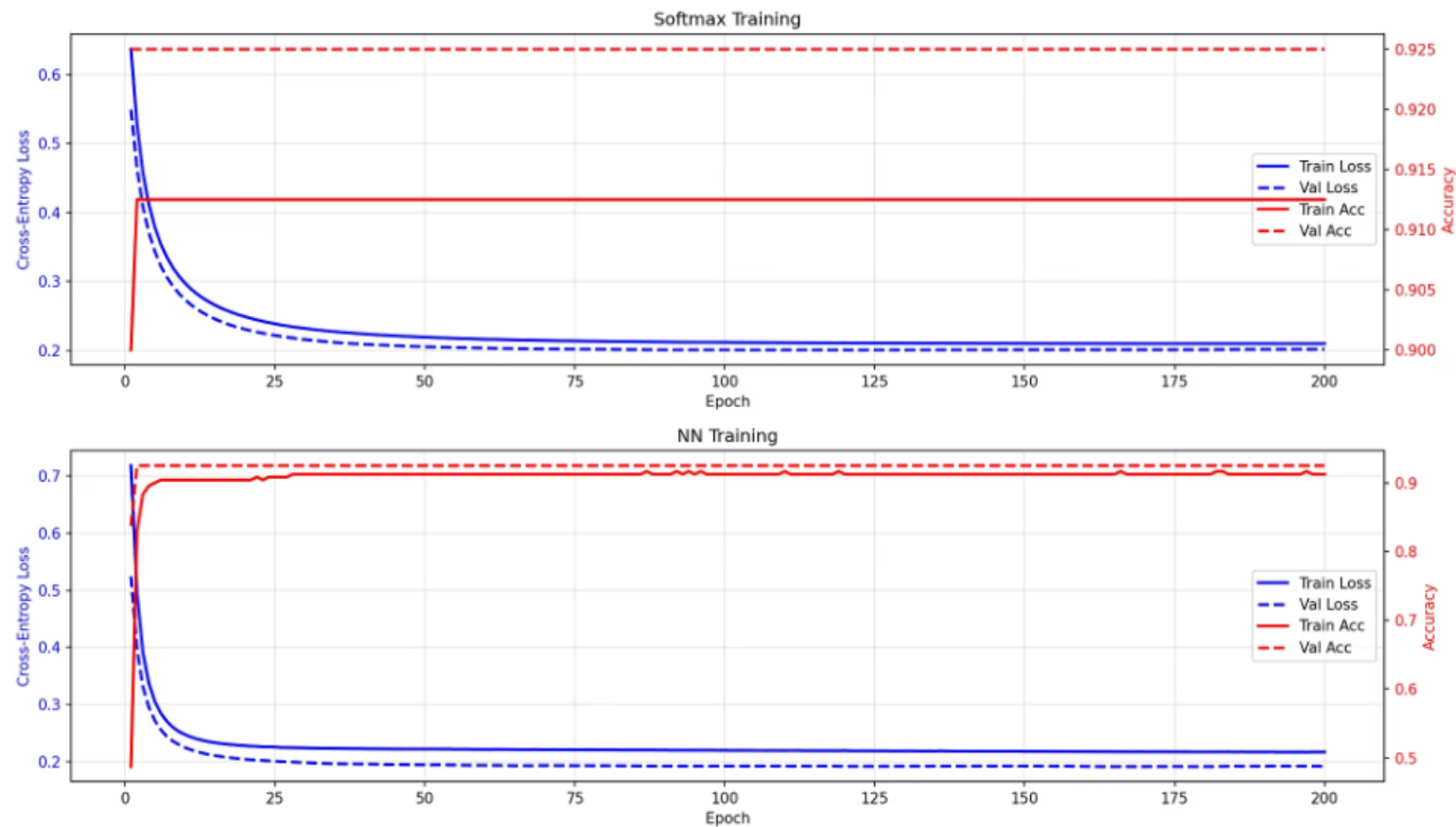
Model	Test Accuracy	Test Cross-Entropy
Softmax Regression	93.8% $\pm$ 0.8%	0.270 $\pm$ 0.012
Neural Network ($h = 32$)	95.3% $\pm$ 0.6%	0.166 $\pm$ 0.009

5 seeds, 95% confidence intervals (t-distribution)

Key Findings:

- Non-overlapping CIs $\rightarrow$ statistically significant improvement
- Loss difference more pronounced than accuracy difference
- Neural network better calibrated

Training Dynamics – Gaussian



Training Dynamics: Linear Gaussian Dataset

Model	Final Train Loss	Final Val Loss	Final Val Accuracy
Softmax Regression	~0.15	~0.15	95.0%
Neural Network (h=32)	~0.16	~0.16	95.0%

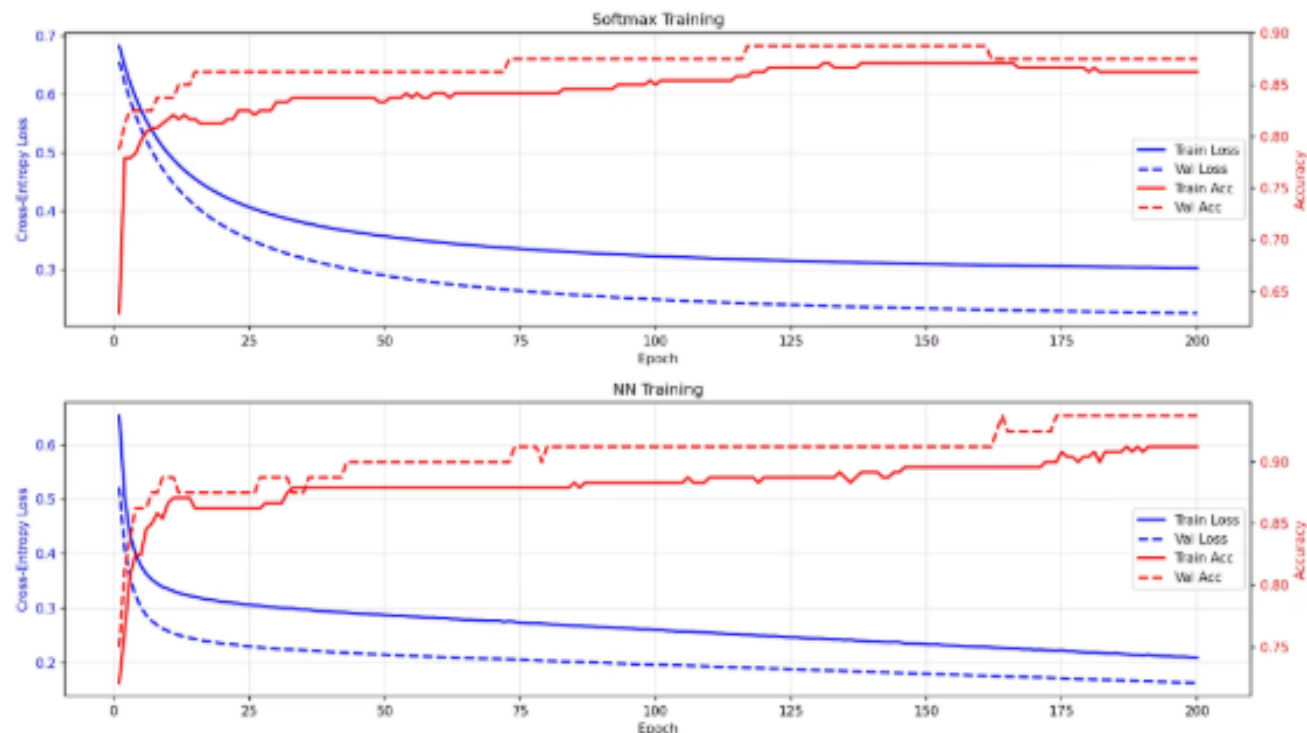
Key Observations:

- Both models converge quickly (within 50-100 epochs)
- Loss curves are nearly identical
- No overfitting – validation loss tracks training loss

Interpretation:

The task is **simple and linear**. Both models learn equally well. The extra capacity of the neural network provides no benefit.

Training Dynamics – Moons



Training Dynamics: Moons Dataset

Comparison: Softmax vs Neural Network

Model	Final Val Loss	Final Val Accuracy
Softmax Regression	0.21	86%
Neural Network	0.18	92%

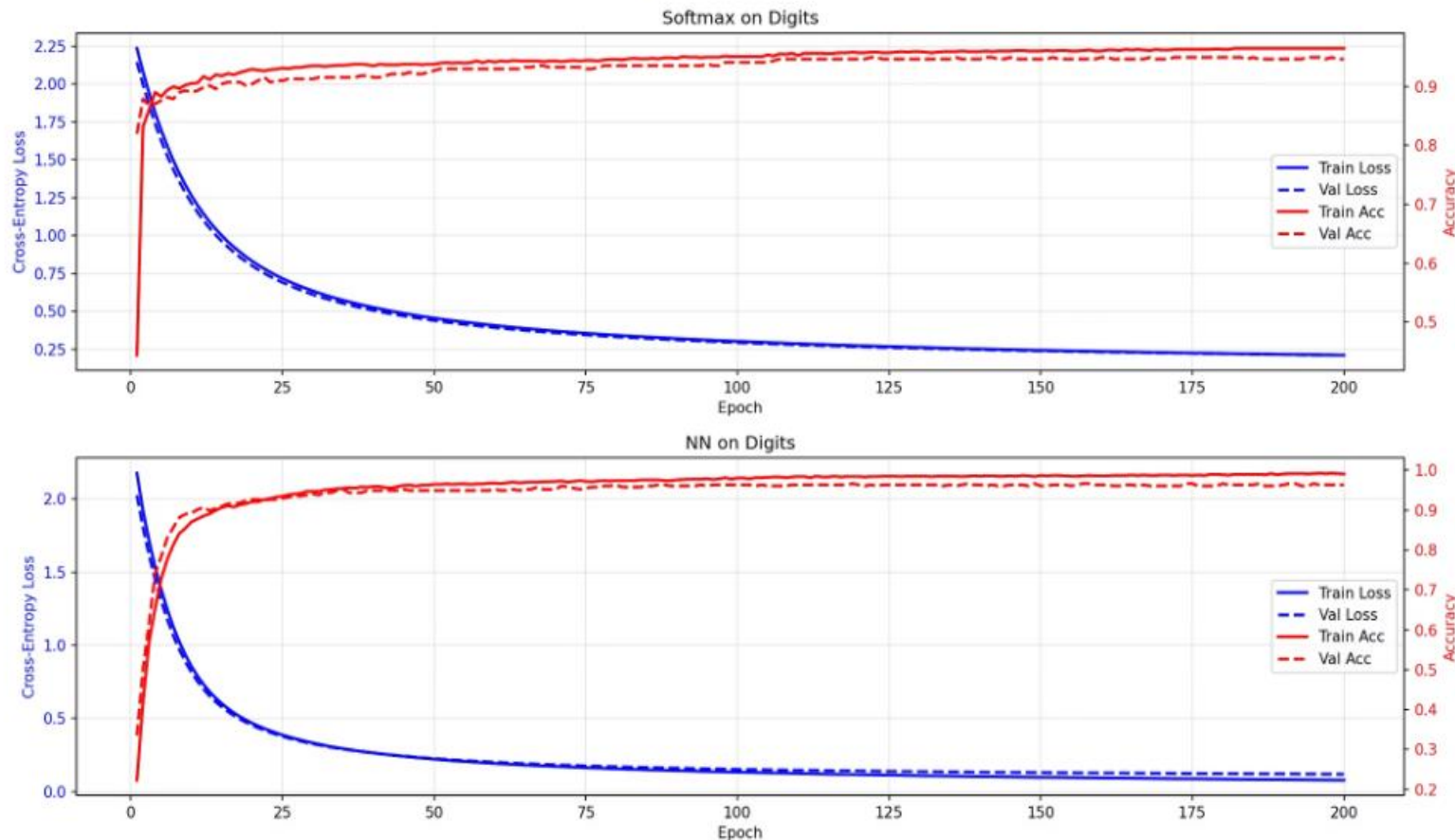
Key Observations

1. Neural network achieves **+6% higher accuracy** (92% vs 86%)
2. Neural network also achieves **lower loss** (0.18 vs 0.21) – unambiguous improvement
3. Softmax at 86% is near its linear ceiling for moons
4. NN at 92% approaches the Bayes error (~93-94% for typical moons noise)

Interpretation

The neural network outperforms softmax on **both metrics** because moons is not linearly separable. Softmax draws a straight line, capping at 86%. The NN learns a curved boundary, reducing both error rate and cross-entropy. The lower loss (0.18 vs 0.21) confirms the NN produces **better-calibrated probabilities**, not just more correct predictions. This is the clean case: nonlinear models dominate linear ones when the data geometry demands it.

Training Dynamics – Digits



Training Dynamics: Digits Dataset

Comparison: Softmax vs Neural Network (h=32)

Model	Convergence Speed	Final Val Loss	Final Test Accuracy
Softmax Regression	Steady	0.271	93.8%
Neural Network (h=32)	Faster	0.166	95.3%

Key Observations:

- Neural network converges faster and achieves lower validation loss
- Softmax loss decreases steadily but plateaus higher
- Both models stable – no divergence or instability

Interpretation:

The hidden layer provides meaningful representational improvement even on real-world data. The lower cross-entropy indicates **better calibrated probabilities**, not just higher accuracy.

Results Summary

Dataset	Softmax	Neural Network	Gap	Insight
Linear Gaussian	95.0%	95.0%	0%	Linear suffices
Moons	85.0%	95.0%	+10%	Nonlinearity necessary
Digits	93.8%	95.3%	+1.5%	Modest but significant

Key patterns:

- Nonlinearity helps when data geometry is curved
- Offers no benefit when boundaries are linear
- Yields modest gains on real-world data with residual nonlinear structure

Limitations

What our study does NOT test:

- Deeper networks or alternative activations (ReLU, etc.)
- Hyperparameter tuning (fixed defaults used)
- Other evaluation dimensions (computational cost, latency, robustness, fairness)
- Repeated seeds for capacity ablation (single runs only)
- Neural network sensitivity to dimensionality reduction
- Cross-validation (fixed splits only)

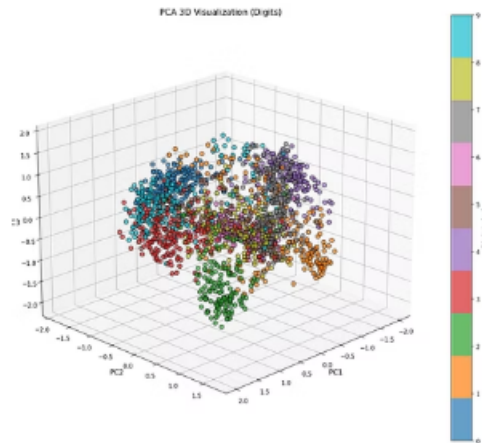
Takeaway: Results should not be overgeneralized beyond the studied architecture and datasets

PCA 3D Visualization

Digits Data: First Three Principal Components

Visualization:

- 3D scatter plot of digits projected onto PC1, PC2, and PC3
- Colors represent digit classes 0-9



Key Observations:

- Digit 0 forms a tight, well-separated cluster (explaining perfect classification)
- Visually similar digits appear close in the low-dimensional space
- High spread clusters represent digits that are written variably
- The 3D projection reveals separation that 2D alone cannot capture

Interpretation:

The digits data lives on a low-dimensional manifold (effective dimension ~10-20). The first three principal components already capture meaningful clustering structure, confirming that linear methods can work reasonably well, but residual nonlinearities remain that the neural network captures.

Confidence Analysis – Softmax Regression

Prediction Confidence: Softmax Regression on Digits Test Set

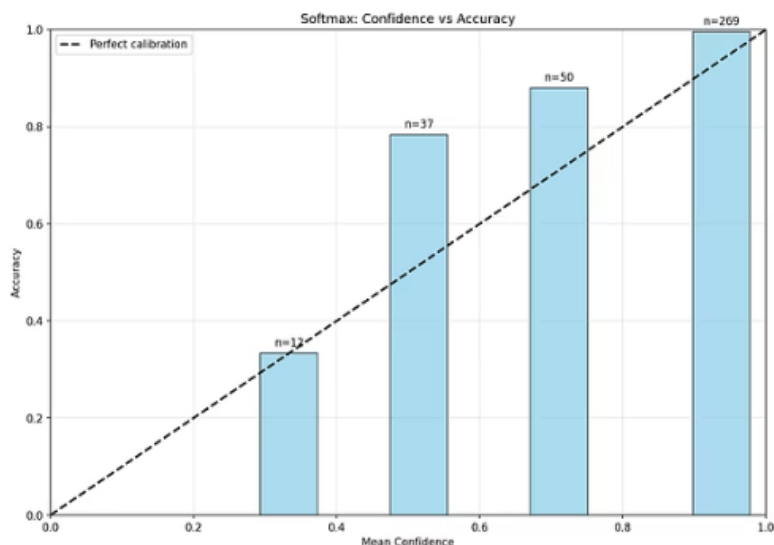
To put simply, confidence is, how sure a model is about its predictions.

Key Observations

- **Systematic underconfidence:** Across most bins, actual accuracy exceeds predicted confidence – the model consistently underestimates its own accuracy
- **Well-calibrated at high confidence:** Predictions above 0.8 are nearly always correct, with actual accuracy reaching 100%
- **Calibration improves with confidence:** The gap between predicted confidence and actual accuracy narrows as confidence increases

Interpretation

Softmax regression produces reasonably well-calibrated probabilities at high confidence levels. However, the model exhibits systematic underconfidence – when it predicts 50% confidence, the true accuracy is substantially higher at 79%.

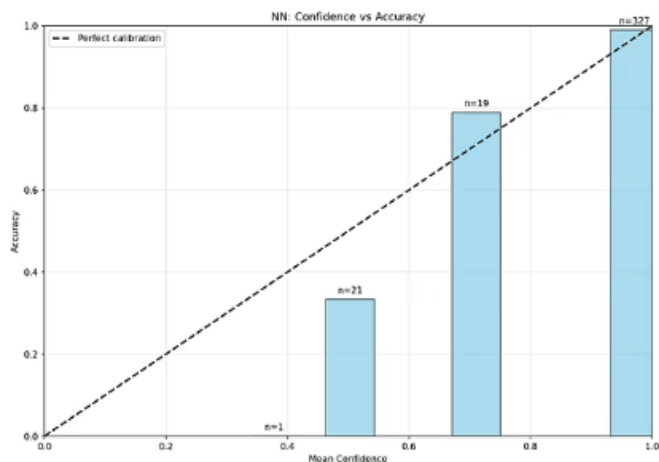


Confidence Analysis – Neural Network

Prediction Confidence: Neural Network (h=32) on Digits Test Set

Key Observations

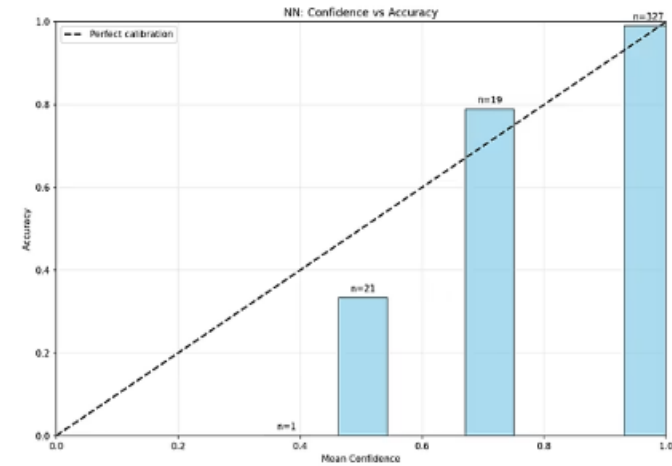
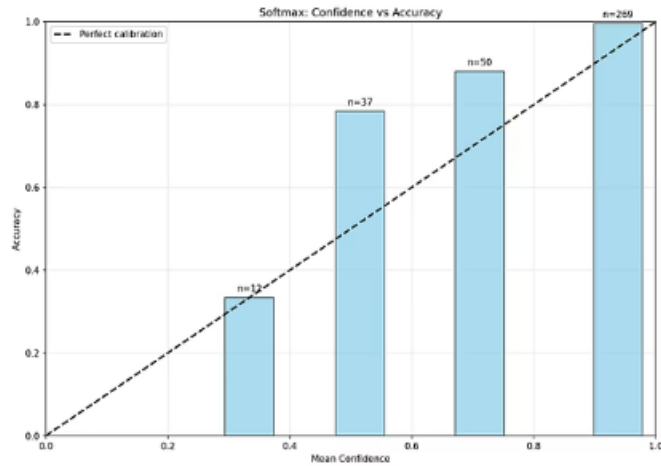
- Empty 0–0.2 bin: No low-confidence predictions
- Highly skewed: 89% of predictions in 0.8–1.0 bin
- Overconfident in low range: 50% confidence, 31% accuracy
- Underconfident in mid-range: 70% confidence, 79% accuracy
- Perfect at high range: 99% accuracy for predictions above 0.8



Interpretation

The model is asymmetrically calibrated: it rarely predicts low probabilities and favors high-confidence positives. Calibration is inconsistent, because overconfident at low confidence, underconfident at mid-range, well-calibrated at high. Probability outputs are distorted, likely from class imbalance or training bias, and require post-hoc calibration.

Confidence Comparison



Conclusions

- **Neural network is more confident:** Average confidence of 0.91 vs 0.87
- **Both are reliable at high confidence:** Predictions above 0.8 are nearly always correct
- **Both models are similarly well-calibrated at high confidence:** the confidence-accuracy gap is small and comparable (0.10 for softmax, 0.09 for neural network).

Overall: The neural network is more confident and marginally better calibrated at the high end, though both models are similarly reliable above 0.8 and miscalibrated at lower confidences.

Conclusions

When does nonlinearity help?

Scenario	Conclusion
Linear geometry	No benefit – linear model is sufficient
Curved geometry	Essential – hidden layer enables curved boundaries
Real-world data	Modest gains – residual nonlinear structure matters
Insufficient capacity	Collapses to linear performance – hard representational ceiling
Excess capacity	Diminishing returns – optimization difficulty

The hidden layer transforms input space via *tanh* activation, enabling curved decision boundaries that linear models cannot produce.



Questions?